Kitchcu

Complete Executive & Engineering Guide - CEO, CPO & CTO

Kitchcu cloud kitchen platform

Audience: CEO, CPO, CTO, Product, Engineering, DBA, QA, Investors, AI agents

Three executive lenses:

- CEO - positioning, subscription economics, GTM, risks, India 1st / world 3rd claim
- CPO - personas, P1-P12 / C1-C6, modules M01-M18 (Definition/How/Why), journeys, KPIs
- CTO - architecture WHY, 100k sessions, TDD+EDD, diagrams, ER, security, build matrix

This guide includes:

- Part 0 definitions + glossary (tenant, outbox, EventEnvelope, live-capture, master order)
- Parts I-III: CEO / CPO / CTO lenses through S18 + GST; E1/E2 design-ready
- Aggregated OpenAPI portal (/openapi.json, /docs, /redoc, portal /openapi) + docs/API.md
- Parts IV-V: product flows (+ delivery payer, super-admin) + UI Catalog (8 JPEGs)
- Addons: dish ready-within, Maps tracking, login highlights, Control plane, refunds
- Full userflows pack: docs/CKAC-USERFLOWS.md / .pdf
- Parts VI-VIII: brand/UX, demo credentials, ports, charter, E1/E2 summary
- Zero per-order food commission; owner-owned CRM; live-capture truth
- India's first - and the world's third - platform with this feature stack
- PDF layout v3.2: running header clearance; no caption/title overlap

Version 3.2 | July 2026 | Confidential

Table of Contents

PART VIII - Forward Design

Appendices

What KitchCu Is

1. What KitchCu Is / Is Not

KitchCu (repo/schema identifiers use legacy short name ckac) is a B2B2C growth operating system for cloud kitchens and home food businesses. It is deliberately not a food-delivery marketplace and not a restaurant point-of-sale system.

KitchCu IS	KitchCu IS NOT
Owner ops hub: orders, menu, GST, CRM	Food aggregator owning the customer
Subscription SaaS for the kitchen	Per-order commission marketplace
Customer discovery + checkout PWA	Restaurant POS / dine-in / KDS / hotel
Multi-kitchen cart, one payment	Single-vendor cart that hides fees
Live-capture-only dish heroes	Stock/studio photography as dish hero
Event-driven microservices + schemas	Monolith or cross-schema writes
Built for 100k concurrent sessions	Prototype that scales later

"Does this help a cloud kitchen grow, without dependence on an aggregator? If no - reject the feature."

Revenue model: Owner Starter / Growth / Pro subscriptions with zero per-order food commission. Billing manages subscriptions and split settlements - never a commission ledger.

2. Glossary Highlights

Term	Definition
Kitchen / tenant	One cloud kitchen; kitchen_id scopes all
Owner / Customer	OTP+JWT; customer JWT type:customer (sep
Live-capture	Hero media must be is_live_capture:true
Order	Single-kitchen purchase + status machine
Master order	MORD-YYYYMMDD-XXXX wraps 2+ sub-orders,
Bill / order code	BILL-YYYYMMDD-SEQ; code = kitchen_code-b
Settlement	Net to kitchen; no KitchCu take-rate / c
Schema ckac_*	One Postgres schema per microservice bou
Gateway	Sole public HTTP edge :18000; path routi
EventEnvelope	Structured fact after every write (ckac_
Redis Stream	ckac:<domain>:<aggregate>; Phase 1 event
Transactional outbox	Domain write + ckac_events.outbox same c
Idempotency-Key	Client header on money POSTs; prevents d
Correlation ID	X-Correlation-ID from gateway across all
Home-taste rating	0.6*taste + 0.4*quality; verified delive
Ready-within / max_time	Honest readiness from dish timing; cart
Delivery payer	Owner pays in-range; customer pays exten
Feature flag	Admin Control kill-switch (e.g. refunds_
Purchase / lock	E1/E2 design: bill-backed stock + recipe

Executive Strategy

1. Executive Summary & Platform Snapshot

KitchCu is a B2B2C cloud kitchen operating system. Owners run orders, menu, quality, CRM, payments, GST, and growth from one PWA. Customers get live-capture honesty, distance-aware delivery fees, home-taste ratings, and multi-kitchen checkout - without KitchCu taking customer ownership or a per-order commission.

Stakeholder	Challenge	KitchCu answer
Owner/chef	WhatsApp chaos, aggregator tax	Unified hub + flat subscription
Customer	Fake photos, opaque fees	Live media + fee quotes + tracking
Platform	Capital-efficient SaaS path	PWA-first + event-driven services

"Keep day one simple: an owner accepts an order and sees revenue the same day. Growth layers unlock only after traction."

S1-S18
Sprints shipped

13
Domain services

Live
GST finance

Design
E1/E2 quality loop

Positioning claim: India's first - and the world's third - platform with this feature stack (ops OS + live-capture trust + multi-kitchen zero-commission checkout + GST + home-taste ratings + growth intelligence in one subscription).

2. Market Positioning & Business Model

```
 Aggregators (marketplace)          KitchCu (operating system)
-------------------------          -------------------------
Per-order commission 18-30%        Flat monthly subscription
Platform owns the customer         Owner owns CRM + customer data
Stock / studio dish photos         Live-capture media, server-enforced
Speed-race delivery timers         Owner-set prep/delivery SLA
Single-kitchen cart only           Multi-kitchen master checkout
No GST / quality OS                GST, ingredients, recipe standards
```

Non-negotiable: zero per-order food commission - a product-design constraint. Settlement math has no take-rate field.

Tier (dev)	Monthly	Unlocks
Starter	Rs 499	Operations + basic reports
Growth	Rs 999	CRM, coupons, deeper insights
Pro	Rs 1,999	Multi-kitchen ops, priority support

Target unit economics: CAC < Rs 2,000 ? LTV > Rs 18,000 ? LTV:CAC > 3:1 ? gross margin > 75%.

3. Go-to-Market Phases & Risks

Phase	Goal	Status
1 Foundation	Owner runs kitchen end-to-end	S1-S18 shipped
2 Growth polish	Offline PWA, CRM automation	Continuous
3 Quality loop	Purchase ledger + chef standards	Design pack ready
4 Scale	National rankings, forecasting	Future

Risk	Mitigation
WhatsApp API policy change	Manual + PWA intake always available
Feature scope creep	MoSCoW + mandatory design pack gate
Rating fraud	Verified delivered purchases only
Multi-kitchen refunds	Atomic sub-orders + independent settleme
Onboarding friction	Kitchen -> dish -> order under 5 minutes

Product

1. Vision, Personas & Principles

Vision: every cloud kitchen scales like a brand - with data, taste standards, and a direct relationship with its own customers - while diners see honest food truth instead of marketplace noise.

Persona	Promise	Surface
Owner / chef	Chaos -> revenue same day	kitchen.kitchcu.in
Customer	Trust + fair fees + home taste	customer.kitchcu.in
Platform admin	Support queue, oversight	admin.kitchcu.in
Guest / market	Brand story, pricing, demo	Portal kitchcu.in

Non-negotiable principles

- Quality over speed - owner-set prep/delivery; never fake 10-min races
- Truth in media - live-capture heroes only; no stock-photo deception
- Owner owns CRM - spend/patterns/coupons never sold cross-tenant
- Progressive complexity - advanced modules unlock after traction
- Minimal diff - every change touches only what the task requires

2. Challenges to Module Solutions

Owner-side (P1-P12)

ID	Challenge	Modules
P1	Orders trapped in WhatsApp	Order ? Notification
P2	Aggregator commission	Billing (subscription)
P3	No daily revenue visibility	Analytics ? Growth
P4	Stock photos erode trust	Catalog live-capture
P5	Taste drifts batch to batch	Recipes ? Ratings ? E2
P6	Customer data on platforms	Marketing CRM
P7	Promotion guesswork	Growth ? Daily menu
P8	Multi-channel lifecycle chaos	Order status machine
P9	Stock-outs mid-service	Ingredients ? E1 design
P10	GST / accountant handoff	Billing GST
P11	Skills / trial dishes hard	Learning ? Community
P12	Engagement without ads	Streaming (opt-in live)

Customer-side (C1-C6)

ID	Challenge	Modules
C1	Cannot trust menu photos	Catalog live-capture
C2	Opaque delivery fees	Delivery
C3	Weak order tracking	Order ? Notify ? Tracking
C4	Generic star ratings	Ratings (taste + quality)
C5	One kitchen per cart	Master checkout ? Split
C6	Hard to find local kitchens	Identity discovery

3. Module Catalog M01-M18

Each module is a bounded product + engineering context. Below: Definition, How it works, and Why / challenge solved. Status reflects code in the monorepo.

M01 - API Gateway

Definition: Single public HTTP edge for all /api/v1/* traffic.

How: Path-prefix router to 13 services; injects X-Correlation-ID; aggregates health; zero business logic.

Why / challenge: Without it four PWAs would hardcode topology. Port 18000. Live.

M02 - Identity & Kitchen Profile

Definition: Auth + tenant-root for owners, customers, kitchens, admins.

How: Owner OTP->JWT; Customer WhatsApp OTP or social OAuth (type:customer). Kitchen gets PostGIS point + code CKPNQ001. Nearby via ST_DWithin.

Why / challenge: Solves P1 bootstrap, C6 discovery. Schema ckac_identity. S1+.

M03 - Catalog & Live Media

Definition: Menu of truth: cuisines, categories, dishes, live-capture heroes.

How: DishMediaInput rejects heroes without is_live_capture:true. Redis cache menu:{kitchen_id} TTL 5 min; invalidate on dish events.

Why / challenge: Solves P4/C1 photo deception. Schema ckac_catalog. S2.

M04 - Ingredients & Recipes (F19)

Definition: Pantry stock, per-dish recipes, prep steps.

How: On order accept (not place), deduct recipe qty x order qty; clamp at 0 with warning. Low-stock thresholds on ingredients.

Why / challenge: Solves P9 stock-outs; foundation for P5/E2. S15. E1 purchase ledger not built.

M05 - Order Operations

Definition: Intake any channel, lifecycle, PDF bills, owner analytics.

How: Status: received->accepted->preparing->ready->out_for_delivery->delivered | cancelled. Immutable order_status_events + order.status.changed.

Why / challenge: Solves P1/P3/P8/C3. Schema ckac_orders. S3+.

M06 - Multi-Kitchen Checkout (F06)

Definition: Cart spanning kitchens, one customer payment.

How: Group by kitchen_id; one master_orders + atomic sub-orders (all-or-nothing); one master receipt; each kitchen sees only its sub-order.

Why / challenge: Solves C5 without commission. S8.

M07 - Billing, Payments & Split Settlement

Definition: Order payments, UPI, subscriptions, Route-style splits.

How: One aggregated payment -> settlements[] net_to_owner per kitchen. Never cache payments/settlements. No take-rate field.

Why / challenge: Solves P2 + fair multi-kitchen payout. S6/S9.

M08 - GST Finance

Definition: GSTIN profile, tax invoices, monthly audit, balance sheet.

How: Delivered orders sync to gst_tax_invoices; close posts immutable gst_monthly_audits snapshot for accountant handoff.

Why / challenge: Solves P10. Live in billing (003_gst).

M09 - Notification & Support

Definition: WhatsApp I/O, status push, tracking nudges, AI chat, tickets.

How: Webhook -> drafts; F45 status WA; F29 interval reminders (not fake ETAs); AI chat escalates to support_tickets.

Why / challenge: Solves P1/C3. Schema ckac_support. S4+S14.

M10 - Marketing, CRM & Coupons

Definition: Owner-owned customer relationship layer.

How: kitchen_customers rollup per kitchen only (no cross-kitchen profile). Coupons + segment promotions from CRM.

Why / challenge: Solves P6/P7. Schema ckac_marketing. S10.

M11 - Ratings & Customer Tips

Definition: Verified home-taste/quality ratings + aggregates + tips.

How: Only delivered + owning customer. overall = 0.6*taste + 0.4*quality. dish_suggestions (F20) accept/reject -> E2 inputs.

Why / challenge: Solves C4/P5 signal. Schema ckac_ratings. S11.

M12 - Growth Intelligence

Definition: Actionable suggestions - not vanity dashboards.

How: Combos (F09), patterns (F10), suggestions (F11), daily menu push (F39). Reads orders/catalog/ratings; writes only ckac_growth.

Why / challenge: Solves P3/P7. S12. E2 chef brief not built.

M13 - Delivery Radius, Fees & Tracking

Definition: Distance-aware fee quotes + shareable tracking links.

How: PostGIS geodesic distance; fee before payment (F27/F28/F31); signed time-bounded tracking token.

Why / challenge: Solves C2/C3. Schema ckac_delivery. S13.

M14 - Learning Portal & Dish Trials

Definition: Curated skills + controlled new-dish experiments.

How: dish_trials -> limited invites -> trial_ratings -> promote to real menu. Trials never pollute live ratings history.

Why / challenge: Solves P11. Schema ckac_learning. S16.

M15 - Community & Chef Rankings

Definition: Recipe rewards + chef league table.

How: shared_recipes, appreciations, reward ledger/redemptions; chef_rankings gated by COMMUNITY_MIN_ORDERS_RANKING.

Why / challenge: Solves differentiation / engagement. S17.

M16 - Live Streaming

Definition: Owner opt-in live prep sessions (LiveKit).

How: kitchen_stream_settings + live_sessions; customer live-now filter (F48).

Why / challenge: Solves P12 + stronger C1 trust. Schema ckac_streaming. S18.

M17 - Website PWAs & Portal

Definition: Four installable React surfaces from one Vite monorepo.

How: portal/customer/kitchen/admin bundles share brand.ts; Workbox offline on customer+kitchen; OwnerPageShell for kitchen command center.

Why / challenge: Distribution without app-store gatekeeping. Continuous.

M18 - Kitchen Quality Loop (E1+E2) - Design Only

Definition: Purchases restock -> recipes consume -> ratings signal -> lock standard.

How: E1: purchase ledger + stock_movements. E2: rules-based chef brief (0.45*vol+0.35*taste+tip+0.20*risk); owner Lock snapshots recipe_standard_versions. Growth orchestrates; Catalog owns write.

Why / challenge: Completes P5/P9. Design pack ready - no production code.

4. Product Journeys & Capability Ladder

Owner day-1

```
Register -> OTP -> JWT -> Create kitchen (geo + code)
 -> Add dish (live hero) -> Optional GST profile
 -> First manual / WhatsApp order -> Accept -> Deduct stock
 -> Same-day revenue report
```

Customer trust -> order -> rate

```
Nearby map -> Menu (live photos) -> Fee quote -> Checkout
 -> Pay (single or multi-kitchen) -> Track -> Delivered
 -> Home-taste rating (+ optional tip)
```

Capability ladder (progressive complexity)

Rung	Unlocks
1	Menu + Orders + Reports
2	CRM + Coupons + Delivery fee quotes
3	Ingredients + GST + Growth suggestions
4	Learning + Community + Live stream
5	Quality loop lock (E1/E2, once shipped)

5. Product KPIs

KPI	Near-term	12-month
Active kitchens	10 pilots	500
Platform orders / day	50	5,000
Owner monthly retention	80%	90%
30-day customer repeat	25%	40%
Locked recipe standards / kitchen	-	>= 3 (post-E2)
GST audits closed on time	-	>= 80% registered

Architecture & Engineering

1. Architecture - Why & How

Why microservices (not for their own sake)

Each domain has a different rate of change and failure domain: a live-stream bug must not take down order accept; a GST migration must not block menu edits. DDD + hexagonal: extract a container when data, ownership, or deploy cadence requires isolation. 13 services exist because 13 bounded contexts shipped.

```
services/<name>/
 app/main.py     # FastAPI, lifespan, CORS, health
 app/routes.py   # thin adapters  -  no business logic
 app/schemas.py  # Pydantic + domain logic (the brain)
 app/models.py   # SQLAlchemy 2.0 async  -  persistence only
 alembic/        # one schema per service
 tests/          # test_schemas, test_*, test_events
```

Why a single gateway

Public clients must never learn internal topology. Gateway is dumb on purpose: path routing, CORS, correlation ID, health aggregation, future rate limits. Zero business logic so it scales independently.

Why schema-per-domain (ckac_*)

One Postgres cluster, hard logical walls: a service cannot INSERT/UPDATE/DELETE outside its schema. Cross-schema reads allowed for ownership/evidence only. Prevents silent re-formation of a monolith without database-per-service ops cost.

Why kitchen_id everywhere

Multi-tenant at the kitchen. kitchen_id on every tenant table, always in WHERE, always in cache keys (menu:{kitchen_id}), always checked against owner ownership before writes. Backbone of never-return-cross-tenant-data.

Why Redis Streams + transactional outbox

Sync fan-out would make order placement fail when Notification is slow. Instead: commit domain row + outbox row in one transaction, then XADD to ckac:<domain>:<aggregate>. Write never loses its event; write never waits on slow consumers. Kafka is Phase-4 target when volume breaks the SLO.

Rule	Why
One BC per container	Independent deploy / failure domain
Schema-per-domain	Isolation without DB-per-service cost
No cross-schema writes	Prevents monolith re-formation
Outbox on every write	Events survive process crashes
Gateway sole public edge	Topology can change without client break
kitchen_id scoping	Privacy + business-integrity guarantee

2. Scale Lens - 100,000 Concurrent Sessions

Every architectural decision is reviewed against: would this survive 100,000 concurrent sessions and multi-tenant growth? Applied at pilot scale so scaling never means a rewrite.

Dimension	Implication
Statelessness	Horizontal replicas; JWT + Redis OTP; no
Caching	Tenant-scoped keys; never cache payments
Indexing	kitchen_id leading column on hot list/fi
Events over sync	No HTTP fan-out from a write to five ser
Idempotency	Idempotency-Key + outbox on money paths
CQRS readiness	Analytics/menu behind cache + dedicated
PWA discipline	Workbox offline; delta refetch not full
Pooling	Async pools per service; never per-reque

SLO	Phase 1	Scale
API p95 read	< 200 ms	< 100 ms
API p95 write	< 500 ms	< 200 ms
Menu PWA load	< 2 s on 4G	< 1 s
Order place E2E	< 3 s	< 2 s

Rejects on sight: N+1 hot paths, unbounded lists, sync fan-out, non-tenant cache keys.

3. TDD + EDD - Rules and Rationale

TDD: RED -> GREEN -> REFACTOR

- RED: failing test first from planning-benchmark acceptance criteria
- GREEN: minimal production code - no speculative frameworks
- REFACTOR: clean with tests; no TODO/dummy/placeholder in prod paths
- Layers: test_schemas (domain) ? test_* (API) ? test_events (streams)
- Coverage: 80%+ touched now; 95%+ target; 100% payment/order state machine

EDD: every write publishes via outbox

- Commit owning schema only (ckac_<domain>)
- EventPublisher.publish(..., session=session) same transaction
- EventEnvelope: event_type, aggregate_*, producer, correlation_id, payload
- Stream: ckac:<domain>:<aggregate>; assert in test_events.py

```
 event = EventPublisher.build(
    event_type="dish.created", aggregate_type="dish",
    aggregate_id=str(dish.id), producer="catalog-service", payload={...})
await publisher.publish(stream_key("catalog", "dish"), event, session=session)
```

TDD guarantees logic is correct. EDD guarantees correctness is visible to the rest of the system without DB coupling. Skipping EDD produces silent drift (stale menu cache, stale rating aggregates).

4. System Architecture Diagram

```
 PWAs :13000-13003
  Portal | Customer | Kitchen | Admin
            |
            v  HTTPS /api/v1 + X-Correlation-ID
     API GATEWAY :18000
     (path routing ? CORS ? health)
            |
  +---------+---------+---------+---------+
  v         v         v         v         v
identity  catalog   order    billing  notification
:18001    :18002   :18003    :18004     :18005
marketing ratings  growth   delivery  learning
:18006    :18007   :18008    :18009     :18010
community streaming
```

```
:18011    :18012
          |
 PostgreSQL 16 + PostGIS (schema-per-domain + ckac_events.outbox)
 Redis 7 (Streams events + tenant cache) ? MinIO/S3 (live media)
```

Rules: one BC per container ? cross-service writes via events only ? cross-schema reads for ownership/evidence ? gateway sole public edge.

5. Event & Data Flow Diagrams

Order + stock + notify

```
 Actor -> Order: place/accept
Order: insert order + items + status_events
Order: outbox + XADD ckac:orders:order (order.placed)
Order -> Catalog (internal): deduct stock on accept
Catalog: XADD ckac:catalog:ingredient (stock.deducted)
Notification consumes order.status.changed -> WA (F45) / tracking (F29)
```

Multi-kitchen payment (F06 / F44)

```
 Cart [Kitchen A, B] -> master_order + 2 atomic sub-orders
  -> one aggregated payment (capture)
  -> settlements[] net_to_owner per kitchen
  -> payment.captured / settlement.created -> master-bill PDF
```

GST monthly loop

```
 GSTIN register -> kitchen_gst_profiles
order delivered -> gst_tax_invoices
monthly report + balance sheet -> Close Audit
  -> gst_monthly_audits (immutable snapshot)
```

Quality loop E1/E2 (design)

```
 Purchase posted (+delta) -> stock_movements -> pantry
Order accepted (-delta)  -> stock_movements
Volume + ratings + tips  -> GROWTH chef_brief
Owner Lock               -> CATALOG recipe_standard_versions
Locked qty x forecast    -> restock plan
```

Core Redis streams (representative)

Stream	Sample events
ckac:orders:order/draft/master	order.placed, status.changed, master_ord
ckac:catalog:dish/ingredient	dish.created, stock.deducted
ckac:billing:payment/gst	payment.captured, gst.audit.closed
ckac:ratings:rating	rating.created, aggregate.updated
ckac:growth:suggestion	suggestion.generated, daily_menu.pushed
ckac:notify:*	whatsapp.received, tracking.reminder.sen

6. ER / Schema Overview

Separate PostgreSQL schemas in one cluster - no cross-schema FKs. Relationships are logical via kitchen_id / entity UUIDs.

```
 ckac_identity:  owners 1--* kitchens; customers 1--* oauth; platform_admins
ckac_catalog:   cuisines--categories--dishes--dish_media
                dishes *--* ingredients (dish_ingredients); prep_steps
                (planned E1/E2) stock_purchases, movements, recipe_standards
ckac_orders:    master_orders 1--* orders 1--* items / status_events; drafts
ckac_billing:   payments 1--* settlements; subscriptions;
                gst_profiles 1--* tax_invoices; monthly_audits
ckac_marketing: kitchen_customers, coupons, promotions
ckac_ratings:   dish_ratings, aggregates, dish_suggestions
ckac_growth:    suggestions, seasonal_patterns
ckac_delivery:  delivery_quotes
ckac_learning:  curated_recipes, dish_trials--invites/ratings
ckac_community: shared_recipes, rewards, chef_rankings
ckac_streaming: stream_settings, live_sessions
ckac_support:   support_tickets, messages
ckac_events:    outbox (+ processed_events inbox target Phase 2+)
```

7. Services, APIs, Security & Standards

Service	Port	Schema	Highlights
gateway	18000	-	/api/v1/* router
identity	18001	ckac_identity	/auth /owners /kitchens
catalog	18002	ckac_catalog	menu dishes ingredients
order	18003	ckac_orders	orders analytics bills
billing	18004	ckac_billing	payments gst webhooks
notification	18005	ckac_support	whatsapp support
marketing	18006	ckac_marketing	CRM coupons
ratings	18007	ckac_ratings	ratings suggestions
growth	18008	ckac_growth	/growth/*
delivery	18009	ckac_delivery	quotes tracking
learning	18010	ckac_learning	/learning/*
community	18011	ckac_community	/community/*
streaming	18012	ckac_streaming	/stream/*

Security posture

- JWT Bearer on owner/customer routes; admin JWT platform-scope only
- kitchen_id tenant filter in domain layer on every mutating call
- Pydantic validation; SQLAlchemy parameterized queries only
- Secrets via env / .env.example; never commit secrets
- PII masked in logs; OTP/tokens never logged
- Internal calls: X-Internal-Key (distinct from public JWT)
- /health/live + /health/ready every service; correlation ID from gateway
- Rate limiting at gateway: named Phase-2 target

Aggregated OpenAPI & API Reference (new in 3.1)

The gateway does not hand-write a contract - it fetches each of the 12 domain services' own /openapi.json and merges them (openapi_aggregate.py): schemas/refs get a service prefix, operations get a service-prefixed tag, x-kitchcu-service traces every route back to its owning service. Merged doc is cached; pass ?refresh=true to force a re-fetch after a route change without restarting.

Surface	URL	Purpose
Aggregated spec	gateway /openapi.json	Merged OpenAPI 3.x, namespaced
Swagger UI	gateway /docs	Interactive explorer
ReDoc	gateway /redoc	Read-only reference
Portal explorer	portal /openapi (+/api-docs)	Same schema inside portal shell

Surface	URL	Purpose
Human index	docs/API.md	Auth cheat-sheet + quick-start examples

Route docs are mandatory, not auto-only: every route needs an explicit summary/description/responses= (ckac_common.openapi helpers for 400/401/403/404/409/422 shapes) and every Pydantic field needs Field(..., description=...). An auto-generated schema with no summaries does not satisfy the engineering standard, because the aggregated /docs is the actual integration surface partners read.

8. Build Status Matrix

Module	Sprint	Status
Gateway / Identity / Catalog / Order / N	S1-S4	Done
PWAs + checkout + analytics	S5	Done
Billing + subscriptions	S6	Done
GST profiles/invoices/audit	billing ext	Done
Discovery F32 / history F33	S7	Done
Multi-kitchen F06	S8	Done
Split settlement F44	S9	Done
CRM / coupons / promotions	S10	Done
Home-taste ratings	S11	Done
Growth intelligence + daily menu	S12	Done
Delivery radius/fees/tracking	S13	Done
Tracking reminders + WA updates	S14	Done
Ingredient mapper F19	S15	Done
Learning + dish trials	S16	Done
Community + chef rankings	S17	Done
Live streaming LiveKit	S18	Done
E1/E2 purchases + chef lock	S19 proposed	Design only

Product Flows Step-by-Step

Each flow follows: Actor -> API -> Domain -> DB (tenant) -> Outbox/Event -> Consumers -> UI reflection. For the exhaustive journey pack (every persona, every screen state, every API call) see docs/CKAC-USERFLOWS.md / .pdf - this part stays condensed for inline readability.

1. Owner Onboarding

```
 1. kitchen.kitchcu.in phone -> POST /auth/otp/request
2. OTP (dev 123456) -> POST /auth/otp/verify -> Owner JWT
3. POST /kitchens (name, address, geo)
   Identity: PostGIS point + code CKxxxnnn + kitchen.created (outbox)
4. PWA shows kitchen profile; guide to first dish (rung 1)
```

2. OTP Login (Owner & Customer)

Step	Owner	Customer
1	Phone on kitchen PWA	WA number or social OAuth
2	POST /auth/otp/request	POST /customers/otp/request
3	OTP 123456 (dev)	OTP or OAuth callback
4	verify -> Owner JWT	verify -> Customer JWT
5	Owner session key	Distinct customer session key

JWTs are structurally distinct - owner token rejected on customer routes and vice versa.

3. WhatsApp / Manual Order Intake

```
 Customer WA message -> Meta webhook -> notification
  -> internal POST orders/from-whatsapp (X-Internal-Key)
  -> order_drafts + order.draft.created
Owner confirms in Kitchen PWA -> orders + items status=received
  -> order.placed
Manual path: New Order UI skips parser; same order.placed converge.
```

4. Customer Checkout - Single Kitchen

- Browse menu (live photos) -> cart
- POST /delivery/quote - fee shown before payment (closes C2)
- Pay online/UPI or COD; POST /orders (customer JWT)
- received -> owner accepts -> stock deduct -> lifecycle -> delivered
- F45 WA status + F29 tracking nudges on transitions

5. Customer Checkout - Multi-Kitchen

```
 Cart lines grouped by kitchen_id
POST /orders/master -> master_order + atomic sub-orders
  If any sub-order invalid -> roll back entire master (nothing created)
One payment -> independent settlements
Kitchen A never sees Kitchen B was in the same cart
```

6. Payment & Split Settlement

- POST /billing/payments - one payments row (aggregated if master)
- On capture: settlements[] net_to_owner per kitchen (Route semantics)
- No commission percentage withheld
- Events: payment.captured, settlement.created
- Customer: one master receipt; kitchen: own settlement only

7. GST Monthly Close

```
 Register GSTIN -> kitchen_gst_profiles
delivered orders -> gst_tax_invoices (from registration onward)
Review report + balance sheet
POST .../gst/monthly-audits/close -> immutable gst_monthly_audits
```

8. Ratings After Delivery

```
 order.status.changed -> delivered
Customer: home_taste + quality (+ optional tip)
POST .../ratings (verifies delivered ownership)
recompute aggregate 0.6*taste + 0.4*quality
rating.created / rating.aggregate.updated
Tips -> dish_suggestions pending; accepted tips feed E2 (design)
```

9. Delivery Payer Modes & Maps Tracking

In-range (distance <= max radius): mode self|platform, payer=owner, customer fee 0. Extended: payer=customer (self fee or platform courier quote). Adapter: delivery/app/platform_courier.py. Order stores delivery_mode, delivery_payer, owner_delivery_cost, customer lat/lng. Track + Order Detail: Google Maps directions kitchen->customer. Design: DELIVERY-PAYER-MODE-DESIGN.md.

- Dish timing: prep_time_min + delivery_time_min + max_time_min -> ready-within
- Cart/checkout uses max projected_ready_min across line items
- Owner PATCH /orders/{id}/delivery-fulfillment for self vs platform
- CommissionAdvantagePanel on owner home: 0% food take vs typical aggregators

10. Super Admin Control Plane

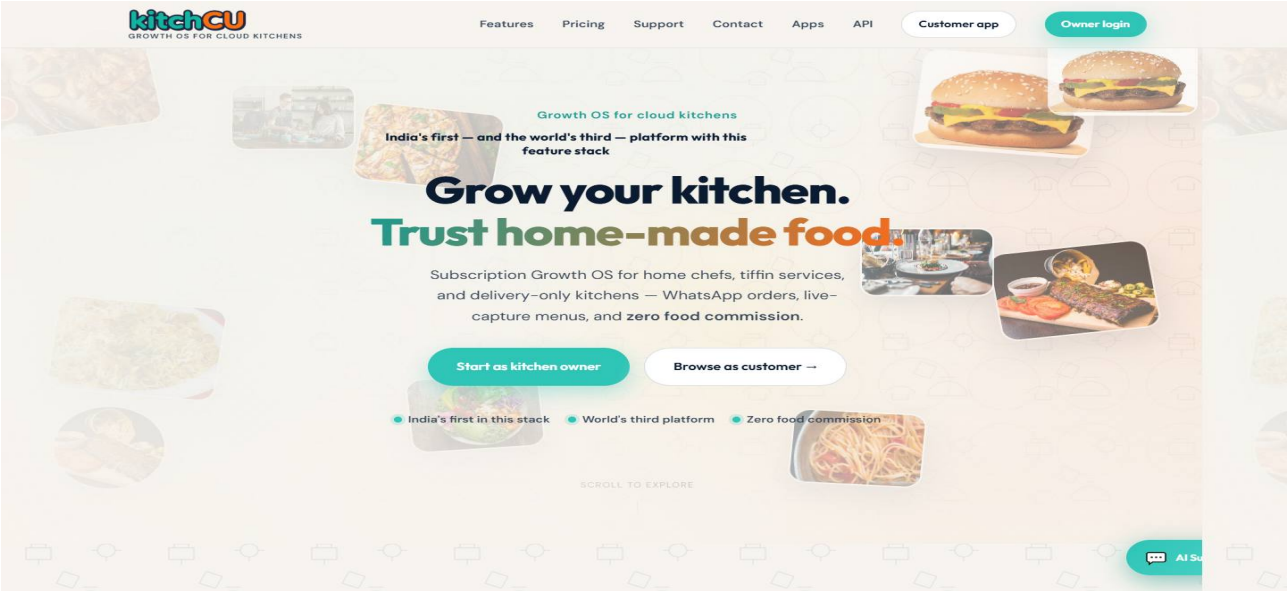
- Login highlights: customers/refunds, flags & journeys, suspend, money, SaaS oversight
- Nav: Overview, Kitchens, Owners, Customers, Orders, Refunds, Tickets, Control
- Control: application data journeys + feature_flags kill-switches + subscription overrides
- Billing admin (refunds/payments/settlements/money-stats) proxied before identity catch-all
- Admin JWT never mutates owner menu/order routes

UI Catalog - Eight Reference Surfaces

Screenshots from docs/assets/ui/ (*-pdf.jpg). Anatomy, UX intent, brand theme, and addon context (login highlights, Control plane, Maps/timing messaging). Layout: caption above image; content starts below running header (no overlap).

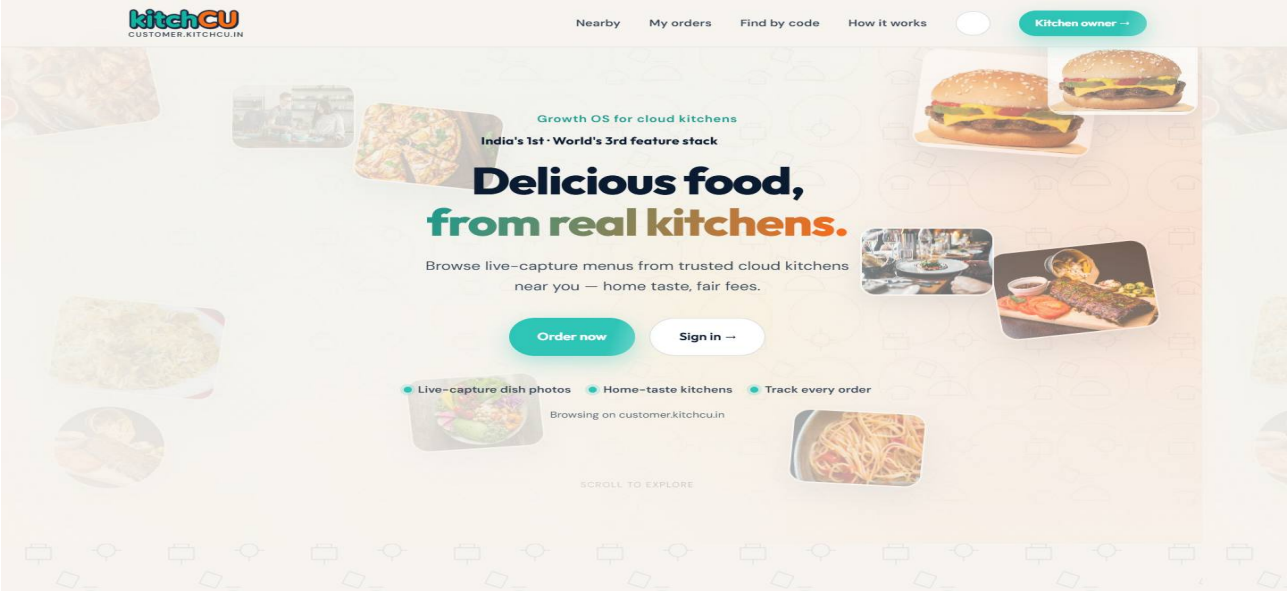
1. Portal Home

Portal (kitchcu.in :13000) - Light marketing. Hero + pricing + live-capture showcase. Persuasion only; reach pricing in two scrolls. Cream #FFF8EE.



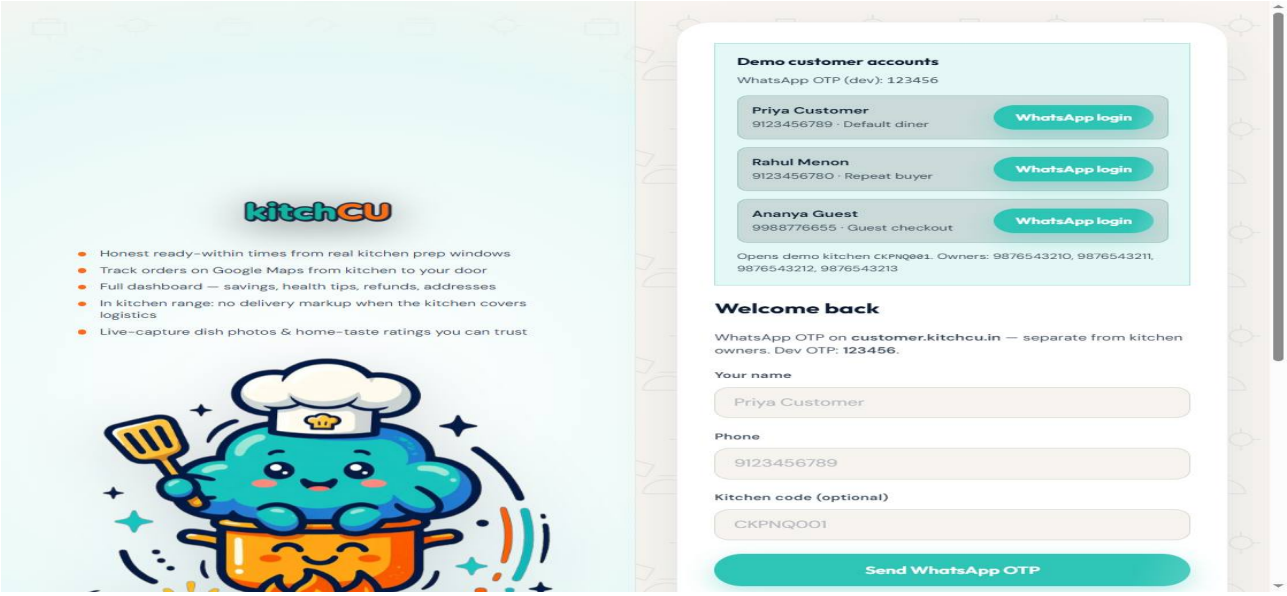
2. Customer Home

Customer home (:13001) - Discovery: NearbyKitchensList, diet/live-capture filters, Dashboard nav. Closes trust + near-me (C1/C6). Demo: CKPNQ001 Pune.



3. Customer Login

Customer login - Left AuthLoginHighlights: ready-within, Maps track, full dashboard, in-range no markup, live-capture. Right: demo WhatsApp OTP + OAuth. Dev OTP 123456.

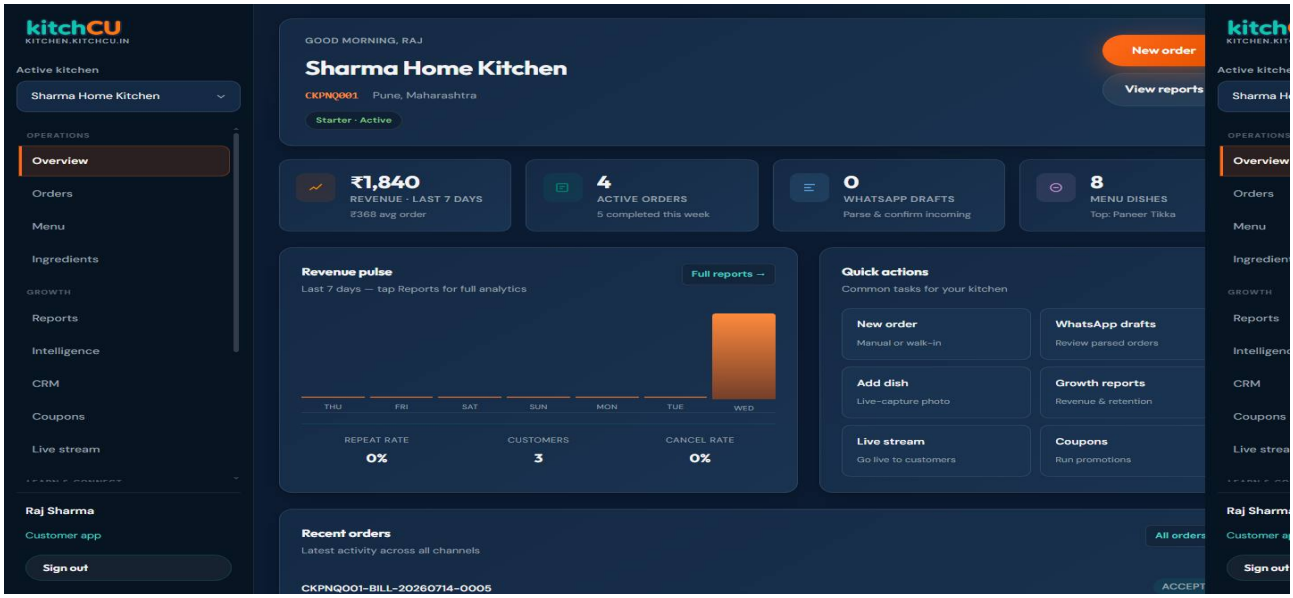


4. Kitchen Login

Kitchen login - Left highlights: zero commission vs 25-30%, dish timing, in-range owner pays / extended customer pays, Maps. Right: demo owner OTP. 9876543210 / 123456 after seed.

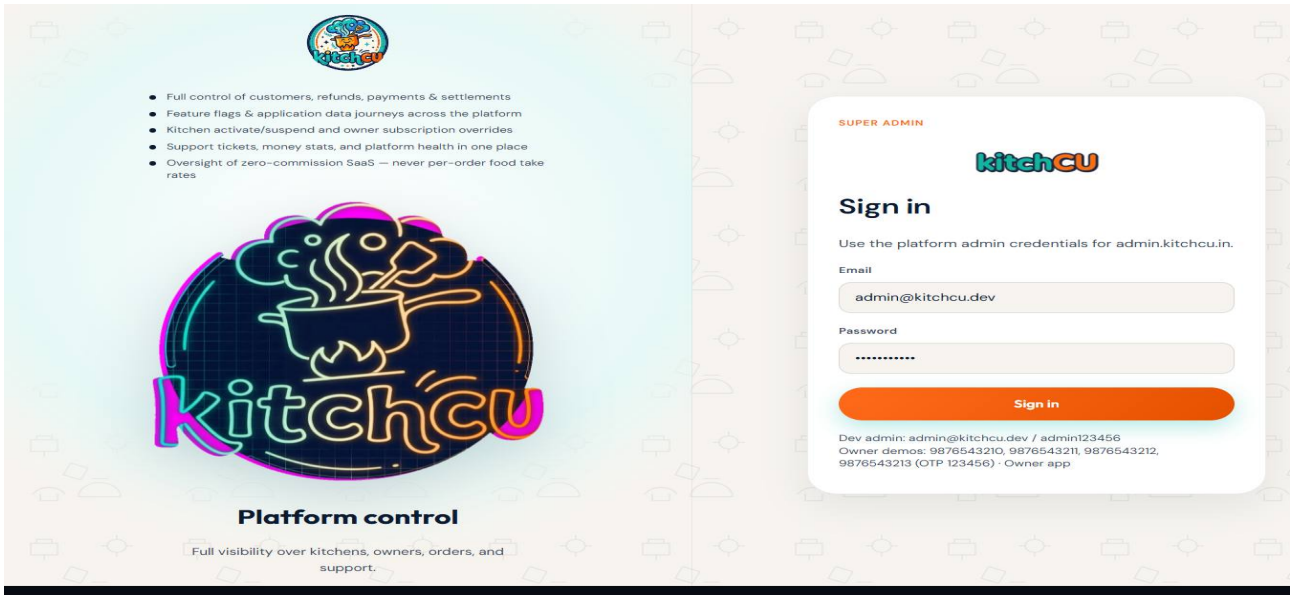
5. Owner Dashboard

Owner dashboard - Dark ops. Side nav capability ladder; New Order; recent orders; CommissionAdvantagePanel (0% food commission). Navy #0B1B32.



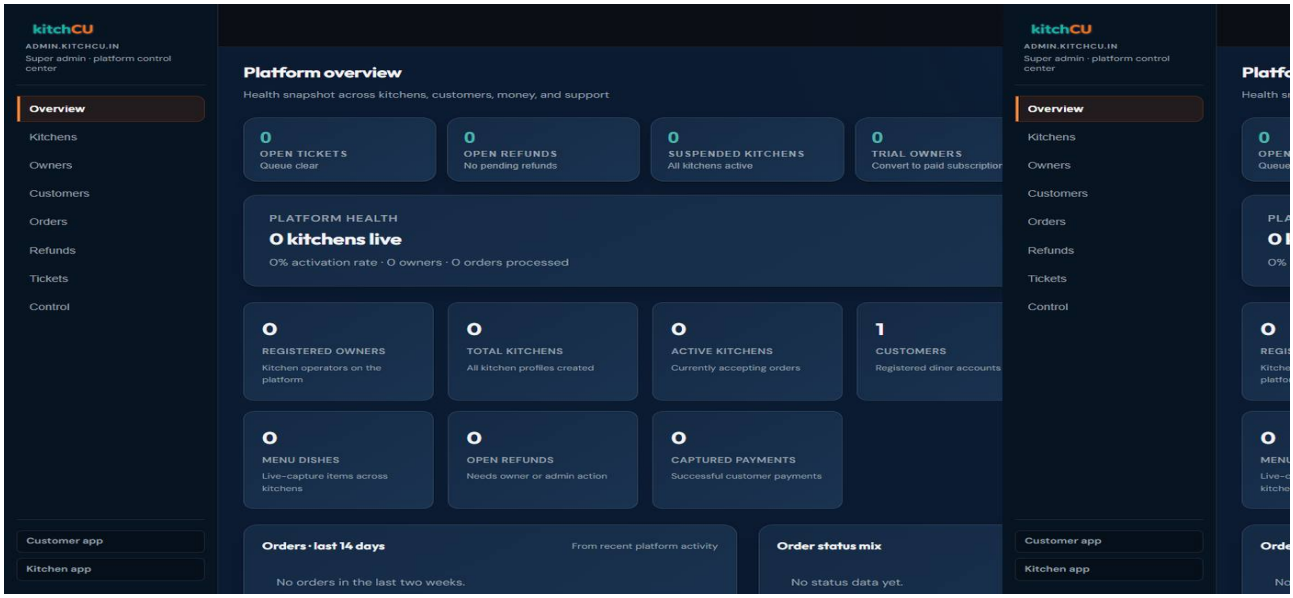
6. Admin Login

Admin login - Platform-control highlights (customers/refunds, flags & journeys, suspend, money, zero-commission oversight). Demo: admin@kitchcu.dev / admin123456.



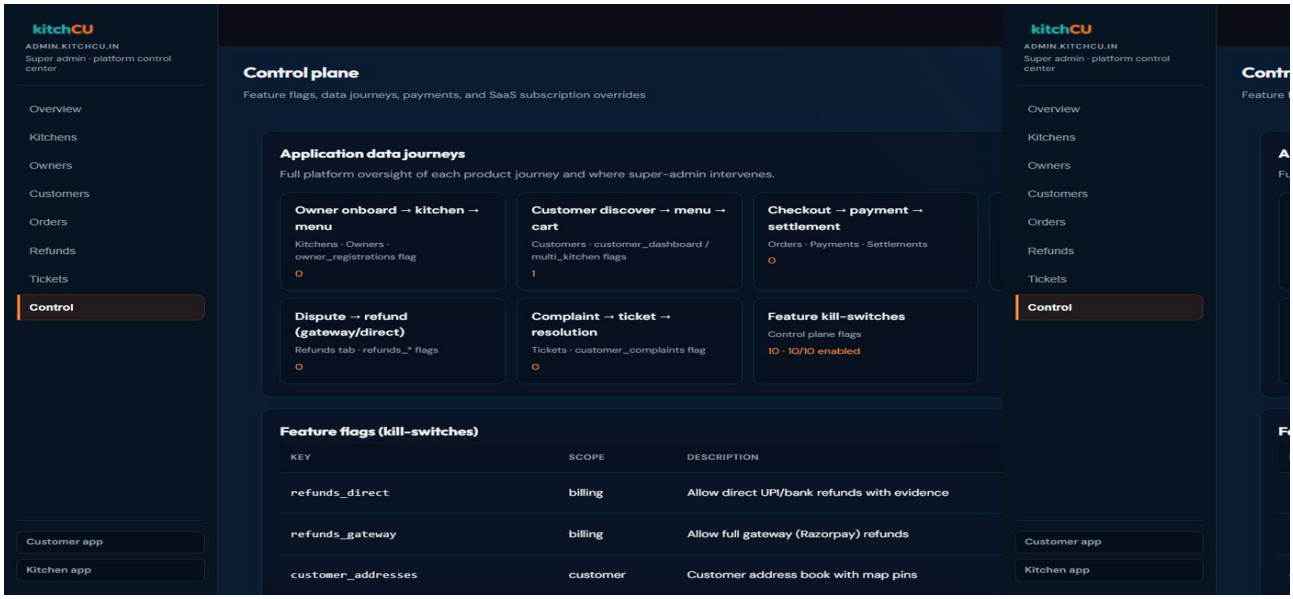
7. Admin Overview

Admin overview - Nav includes Customers, Refunds, Control. Attention tiles, platform health, charts, Quick actions. Platform-scope only; dark ops theme.



8. Admin Control Plane

Admin Control - Application data journeys grid; feature flags table (refunds_gateway/direct, journey keys); subscription overrides; recent payments. Governance without tenant menu mutation.



Brand & UX System

1. Naming, Palette & Themes

UI chrome uses APP_NAME (kitchCU). Art in /logos; served via apps/website/public/brand/; hex + paths only through shared/brand.ts.

Token	Hex	Role
Orange	#FF6B1A	Primary CTA; CU half of wordmark
Teal	#2EC4B6	Secondary; kitch half; success
Navy	#0B1B32	Dark-ops surfaces
Flame yellow	#FFC107	Highlights / attention
Cream	#FFF8EE	Light marketing / customer surfaces

Two themes, one brand

Theme	Where	Why
Light cream/teal/orange	Portal, Customer	Warmth for discovery & appetite
Dark navy-first ops	Kitchen, Admin	Low glare for long service sessions

Same wordmark, mascot, shape language - theme is a UI-mode choice, not a re-brand. Motion: 2-3 intentional moments per flow, never ambient noise. Dish heroes are trust artifacts, never decoration.

Asset map

- wordmark.png - navbars; appicon.png - PWA icons; badge/mascot/mark-circle
- lockup-dark.png - dark chrome; creative-chef - auth; creative-hero - portal
- creative-neon.png - promotional accent

Unified form spacing system (new in 3.1)

owner-forms.css defines one spacing system for every input/select/textarea/label/button across auth (login) and every owner/admin dashboard form - never hand-set per-field margins again.

Token	Value	Role
--kc-field-stack-gap	1.25rem	Field-to-field gap in a form column
--kc-label-control-gap	0.5rem	Label text to its input/select/textarea
--kc-btn-min-h	2.85rem	Submit/primary button min height (44px+
--kc-field-h	3rem	Text input / select minimum height

Spans both brand themes (light auth cards, dark ops dashboards) because rhythm is a UX invariant even where color/theme is not.

Operating Reference

1. Demo Credentials

Role	ID	Secret	Notes
Owner primary	9876543210	OTP 123456	CKPNQ001 Sharma Pune
Owner	9876543211	OTP 123456	Mehta Tiffins
Owner	9876543212	OTP 123456	Desai Cloud Kitchen
Owner	9876543213	OTP 123456	Kulkarni Home Food
Customer	9123456789	OTP 123456	Default diner
Customer	9123456780	OTP 123456	Repeat/VIP segment
Customer	9988776655	OTP 123456	Guest path
Admin	admin@kitchcu.dev	admin123456	Platform scope only

Dev OTP fixed at 123456. Seed: scripts/seed-dev-data.py, bulk_demo_data.py; frontend source of truth: apps/website/src/shared/demo.ts.

2. Ports Table

Surface / Service	Port	Kind
Portal / Customer / Kitchen / Admin	13000-13003	PWA
Gateway	18000	API edge
identity..streaming	18001-18012	Services
PostgreSQL 16 + PostGIS	15432	Data
Redis 7	16379	Cache/events
MinIO API / Console	9000 / 9001	Media

3. Operating Charter Reference

Every change - human or AI - is bound by the always-on KitchCu Strict Operating Charter (.cursor/rules/kitchcu-executive-operating-charter.mdc). Contributors reason as CEO ? CPO ? CTO ? Full-Stack ? UX ? DBA ? QA Lead. Role gates must pass before code: growth/unit economics + zero commission; clear flow + trust; service boundaries + events + no cross-schema writes; minimal diff; brand-first theme; Alembic + tenant isolation; failing tests first + 100% on payment/order state machine. If any gate fails: stop, redesign, do not implement.

Forward Design - E1 + E2

1. E1 + E2 Kitchen Quality Loop (Design, Not Built)

Full spec: docs/E1-E2-KITCHEN-QUALITY-LOOP-DESIGN.md. Why ship together: E1 alone gives accurate stock without taste consistency; E2 alone proposes standards without a trustworthy pantry for restock forecasts. Proposed S19 vertical slice.

```
 Purchases restock pantry (E1)
        |
        v
Recipes consume stock on accept (F19  -  shipped)
        |
        v
Orders + home-taste ratings accumulate signal
        |
        v
Daily chef brief proposes winning standards (E2)
        |
        v
Owner locks standard -> recipe source of truth
        |
        v
Next purchase plan uses locked qty x forecast (E1 <- E2)
```

Decision	Choice	Rationale
New inventory service?	No for S19	Stock already Catalog-owned
Auto-lock overnight?	No	Owner must explicitly Lock
Deduct when?	On accept	Matches F19; no regression
Chef-brief trigger	Owner-initiated	Testable; no silent writes
Scoring engine	Rules, no LLM v1	Auditable explainable formula

Planned tables (ckac_catalog): stock_purchases, stock_purchase_lines, stock_movements, recipe_standard_versions. Events: purchase.posted/voided, recipe.standard.locked/unlocked, suggestion.chef_standard.generated. Approval gate: CPO ? CTO ? DBA ? QA - until all tick, documentation only.

APPENDIX | PART 9

Reference

1. Feature bands F01-F48

Band	Features	Status
Order intake & lifecycle	F01-F05	Done (WA AI partial)
Multi-kitchen & payments	F06, F42-F44	Done
Analytics & growth	F07-F12, F39	Done
Catalog & trust media	F13-F15	Done
Ratings	F16-F18, F20	Done (F20 UI thin)
Ingredients	F19	Done (E1 extends)
Social & subscription	F25-F26	Done
Delivery / discovery	F27-F33	Done
Marketing	F34-F41	Core done
Learning / community	F21-F24	Done
Notifications	F45	Done
Live	F46-F48	Done
GST finance	billing extension	Done
Quality loop	E1, E2	Design only

Full acceptance criteria: docs/CKAC-COMPLETE-PLANNING-BENCHMARK.md

2. Document index

Document	Role
CKAC-COMPLETE-GUIDE.md/.pdf v3.2.3	This CEO/CPO/CTO encyclopedia
CKAC-USERFLOWS.md/.pdf	Full step-by-step user journey pack
API.md	Public API reference + OpenAPI URLs
E1-E2-*-DESIGN.md	S19 quality-loop design pack
CKAC-IMPLEMENTATION-GUIDE.md	Built features mapped to code
CKAC-ARCHITECTURE-CTO.md	CTO layers + CPO traceability
KITCHCU-ENGINEERING-STANDARDS.md	Engineering constitution
MODULE-DESIGN-PACK.md	Mandatory pre-code template
CKAC-COMPLETE-PLANNING-BENCHMARK.md	F01-F48 acceptance criteria
CKAC-SYSTEM-BENCHMARK.md	Architecture, DB, cache, SLOs
AGENTS.md	Agent/engineer quick spec
docs/assets/ui/	UI reference screenshots (8 surfaces)
DELIVERY-PAYER-MODE-DESIGN.md	Delivery payer + courier rules

3. Document control

v3.2.3 July 2026 - P37-P40 dual referrals; GST Excel/PDF + admin GST; super-admin ops console (orders/tickets/settlements/health); platform i18n (10 locales) + HTML/API-key/login-hint harden. Builds on v3.2 Control plane, ready-within, Maps, UI Catalog (8 surfaces), flows 17.9-17.10.

"KitchCu Complete Executive & Engineering Guide v3.2.3 - Confidential - July 2026. India's first - and the world's third - platform with this feature stack."